

# Prueba de Caja Blanca

---

*“Sistema automatizado de gestión de inventarios para la empresa L.F. Accesorios”*

**Integrantes:**

Joel Tapia

Juan Socasi

Vicente Sánchez

Jonathan Rodríguez

**Fecha:** 2026/06/18

## Requisito funcional N2: Agregar Productos

El programa debe agregar los productos con información importante para el administrador

### 1. CÓDIGO FUENTE

```
75 private void guardar() {
76     try {
77         // --- Validar campos obligatorios (no vacíos) ---
78         String nombre = txtNombre.getText().trim();
79         if (nombre.isEmpty()) {
80             throw new IllegalArgumentException("El nombre es obligatorio.");
81         }
82
83         String stockStr = txtStock.getText().trim();
84         if (stockStr.isEmpty()) {
85             throw new IllegalArgumentException("El stock es obligatorio.");
86         }
87         int stock = Integer.parseInt(stockStr);
88         if (stock < 0) {
89             throw new IllegalArgumentException("El stock no puede ser negativo.");
90         }
91
92         String precioStr = txtPrecio.getText().trim();
93         if (precioStr.isEmpty()) {
94             throw new IllegalArgumentException("El precio es obligatorio.");
95         }
96         double precio = Double.parseDouble(precioStr.replace(',', '.'));
97         if (precio < 0) {
98             throw new IllegalArgumentException("El precio no puede ser negativo.");
99         }
100
101         String descripcion = txtDescripcion.getText().trim();
102         if (descripcion.isEmpty()) {
103             throw new IllegalArgumentException("La descripción es obligatoria.");
104         }
105
106         String modelo = txtModelo.getText().trim();
107         if (modelo.isEmpty()) {
108             throw new IllegalArgumentException("El modelo es obligatorio.");
109         }
110
111         String proveedor = txtProveedor.getText().trim();
112         if (proveedor.isEmpty()) {
113             throw new IllegalArgumentException("El proveedor es obligatorio.");
114         }
115
116         String categoria = txtCategoria.getText().trim();
117         if (categoria.isEmpty()) {
118             throw new IllegalArgumentException("La categoría es obligatoria.");
119         }
120
121         String contacto = txtContactoProveedor.getText().trim();
122         if (contacto.isEmpty()) {
123             throw new IllegalArgumentException("El contacto del proveedor es obligatorio.");
124         }
125
126         String telefono = txtTelefonoProveedor.getText().trim();
127         if (telefono.isEmpty()) {
128             throw new IllegalArgumentException("El teléfono del proveedor es obligatorio.");
129         }
130
131         // --- Campo opcional ---
132         String imagen = txtImagen.getText().trim();
133
134         producto = new Producto(0, nombre, stock, precio, descripcion, imagen, LocalDate.now().toString(),
135             modelo, proveedor, categoria, contacto, telefono);
136         guardado = true;
137
138         JOptionPane.showMessageDialog(this,
139             "Producto guardado con éxito.",
140             "Éxito",
141             JOptionPane.INFORMATION_MESSAGE);
142
143         dispose();
144
145     } catch (NumberFormatException e) {
146         JOptionPane.showMessageDialog(this, "Stock y precio deben ser números válidos.", "Error", JOptionPane.ERROR_MESSAGE);
147     } catch (IllegalArgumentException e) {
148         JOptionPane.showMessageDialog(this, e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
149     }
150 }
151 }
```

private void guardar() {

```
try {
    // --- Validar campos obligatorios (no vacíos) ---
    String nombre = txtNombre.getText().trim();
    if (nombre.isEmpty()) {
        throw new IllegalArgumentException("El nombre es obligatorio.");
    }

    String stockStr = txtStock.getText().trim();
    if (stockStr.isEmpty()) {
        throw new IllegalArgumentException("El stock es obligatorio.");
    }
    int stock = Integer.parseInt(stockStr);
    if (stock < 0) {
        throw new IllegalArgumentException("El stock no puede ser negativo.");
    }

    String precioStr = txtPrecio.getText().trim();
    if (precioStr.isEmpty()) {
        throw new IllegalArgumentException("El precio es obligatorio.");
    }
    double precio = Double.parseDouble(precioStr.replace(',', '.'));
    if (precio < 0) {
        throw new IllegalArgumentException("El precio no puede ser negativo.");
    }

    String descripcion = txtDescripcion.getText().trim();
    if (descripcion.isEmpty()) {
        throw new IllegalArgumentException("La descripción es obligatoria.");
    }

    String modelo = txtModelo.getText().trim();
    if (modelo.isEmpty()) {
        throw new IllegalArgumentException("El modelo es obligatorio.");
    }

    String proveedor = txtProveedor.getText().trim();
    if (proveedor.isEmpty()) {
        throw new IllegalArgumentException("El proveedor es obligatorio.");
    }

    String categoria = txtCategoria.getText().trim();
    if (categoria.isEmpty()) {
        throw new IllegalArgumentException("La categoría es obligatoria.");
    }

    String contacto = txtContactoProveedor.getText().trim();
    if (contacto.isEmpty()) {
        throw new IllegalArgumentException("El contacto del proveedor es obligatorio.");
    }

    String telefono = txtTelefonoProveedor.getText().trim();
```

```
        if (telefono.isEmpty()) {
            throw new IllegalArgumentException("El teléfono del proveedor es obligatorio.");
        }

        // --- Campo opcional ---
        String imagen = txtImagen.getText().trim();

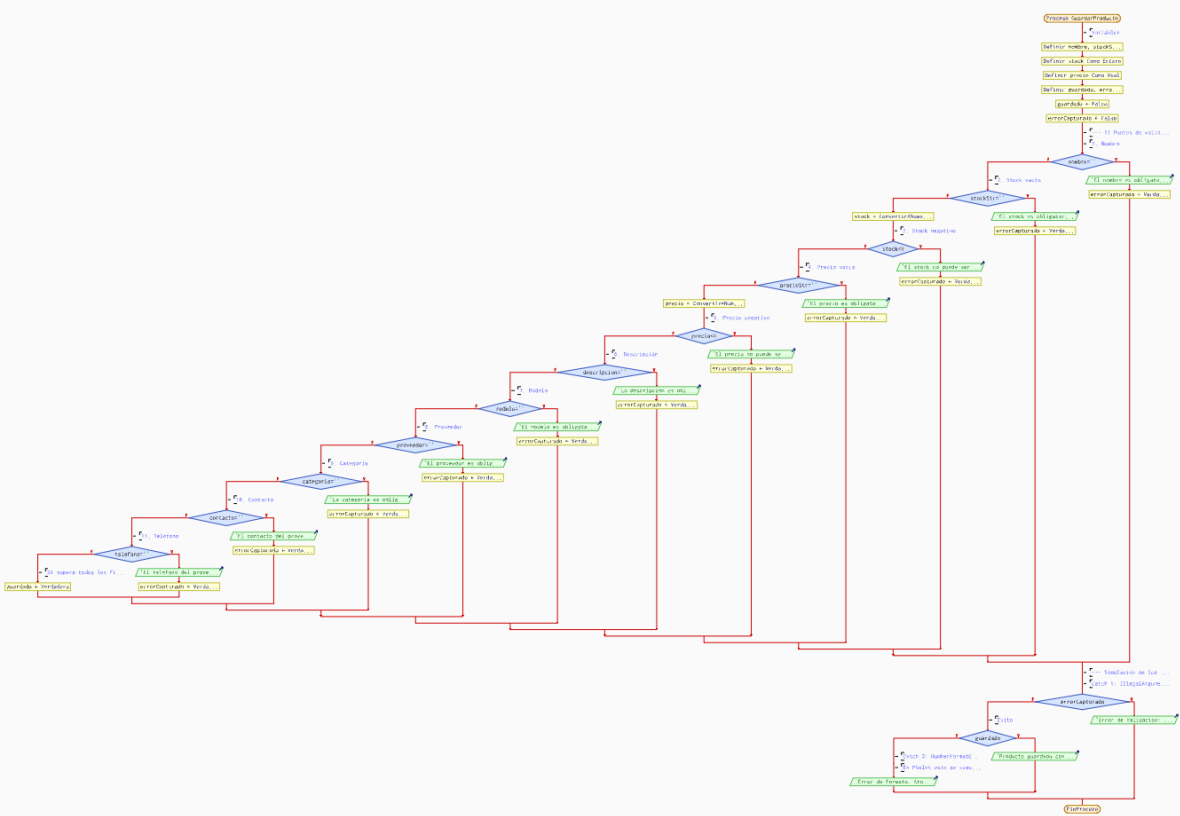
        producto = new Producto(0, nombre, stock, precio, descripcion, imagen,
            LocalDate.now().toString(),
            modelo, proveedor, categoria, contacto, telefono);
        guardado = true;

        JOptionPane.showMessageDialog(this,
            "Producto guardado con éxito.",
            "Éxito",
            JOptionPane.INFORMATION_MESSAGE);

        dispose();

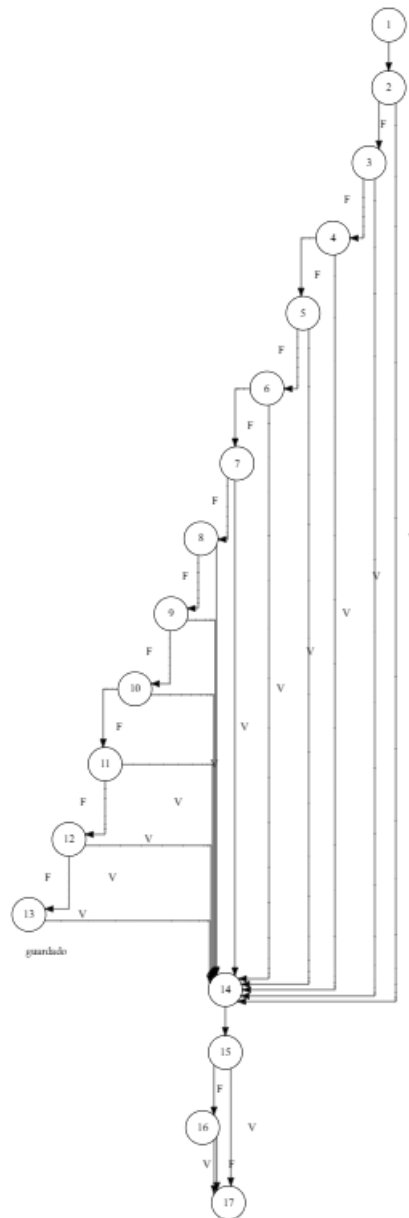
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Stock y precio deben ser números válidos.",
            "Error", JOptionPane.ERROR_MESSAGE);
    } catch (IllegalArgumentException e) {
        JOptionPane.showMessageDialog(this, e.getMessage(), "Error",
            JOptionPane.ERROR_MESSAGE);
    }
}
```

## 2. DIAGRAMA DE FLUJO (DF)



### 3. GRAFO DE FLUJO (GF)

Realizar un GF en base al DF del numeral



#### 4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico)

Determinar en base al GF del numeral 4

RUTAS

| Ruta | Secuencia de nodos                   | Descripción                                                                                                      |
|------|--------------------------------------|------------------------------------------------------------------------------------------------------------------|
| R1   | 1 → 2 → 14 → 15 → 17                 | El campo <b>nombre</b> está vacío. Se activa la bandera de error, se captura en el nodo 15 y termina el proceso. |
| R2   | 1 → 2 → 3 → 14 → 15 → 17             | El campo <b>stock</b> está vacío. Falla la segunda validación, va al nodo de error y termina.                    |
| R3   | 1 → 2 → 3 → 4 → 14 → 15 → 17         | El <b>stock</b> ingresado es negativo. Falla la tercera validación y termina.                                    |
| R4   | 1 → 2 → 3 → 4 → 5 → 14 → 15 → 17     | El campo <b>precio</b> está vacío. Falla la cuarta validación y termina.                                         |
| R5   | 1 → 2 → 3 → 4 → 5 → 6 → 14 → 15 → 17 | El <b>precio</b> ingresado es negativo. Falla la quinta validación y termina.                                    |

|            |                                                                                    |                                                                                                                                                                                     |
|------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>R6</b>  | 1 → 2 → 3 → 4 → 5 → 6 →<br>7 → 14 → 15 → 17                                        | El campo <b>descripción</b> está vacío. Falla la sexta validación y termina.                                                                                                        |
| <b>R7</b>  | 1 → 2 → 3 → 4 → 5 → 6 →<br>7 → 8 → 14 → 15 → 17                                    | El campo <b>modelo</b> está vacío. Falla la séptima validación y termina.                                                                                                           |
| <b>R8</b>  | 1 → 2 → 3 → 4 → 5 → 6 →<br>7 → 8 → 9 → 14 → 15 → 17                                | El campo <b>proveedor</b> está vacío. Falla la octava validación y termina.                                                                                                         |
| <b>R9</b>  | 1 → 2 → 3 → 4 → 5 → 6 →<br>7 → 8 → 9 → 10 → 14 → 15<br>→ 17                        | El campo <b>categoría</b> está vacío. Falla la novena validación y termina.                                                                                                         |
| <b>R10</b> | 1 → 2 → 3 → 4 → 5 → 6 →<br>7 → 8 → 9 → 10 → 11 → 14<br>→ 15 → 17                   | El campo <b>contacto</b> está vacío. Falla la décima validación y termina.                                                                                                          |
| <b>R11</b> | 1 → 2 → 3 → 4 → 5 → 6 →<br>7 → 8 → 9 → 10 → 11 → 12<br>→ 14 → 15 → 17              | El campo <b>teléfono</b> está vacío. Falla la undécima validación y termina.                                                                                                        |
| <b>R12</b> | 1 → 2 → 3 → 4 → 5 → 6 →<br>7 → 8 → 9 → 10 → 11 → 12<br>→ 13 → 14 → 15 → 16 →<br>17 | <b>Éxito:</b> Todos los campos son válidos. Pasa por todos los filtros, activa la bandera guardado (nodo 13) y el mensaje de éxito (nodo 16 por el camino Verdadero).               |
| <b>R13</b> | 1 → [Nodos 2 al 12] → 14 →<br>15 → 16 → 17                                         | <b>Ruta estructural (Catch 2):</b> Camino lógico para evaluar el escenario donde guardado = Falso en el nodo 16 (simulando el NumberFormatException o fallo de formato silencioso). |

## 5. COMPLEJIDAD CICLOMÁTICA

- **Número de nodos (N):** 17
- **Número de nodos predicados/decisión (P):** 13 (Nodos 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 15, 16)
- **Número de aristas (A):** 29

Se puede calcular de las siguientes formas:

V/G) Nodos predicados (IF-THEN-ELSE) +1

$V(G) = 13 + 1$

$V(G) = 14$

$V(G) = A - N + 2$

$V(G) = 29 - 17 + 2$

$V(G) = 14$

DONDE:

P: Número de nodos predicado

A: Número de aristas

N: Número de nodos

